Wiki.js on AWS

One module at a time

Terraform workshop — day 2 lab

What you'll have in two hours

```
browser ──► load balancer ──► Wiki.js (container) ──► PostgreSQL
```

- A real app with a real UI: **Wiki.js**, a wiki you'll log into and write pages in
- A managed database (**RDS PostgreSQL**) whose password nobody ever types
- A **module** you wrote, used to deploy it

Why a module?

- Your next service needs the same things: load balancer, security groups, logs, a role, a task, a service
- Copy-paste works once. The third copy has a bug the first one doesn't
- A module turns "how we run a web service here" into **one reviewed, reusable decision**
- Today: we design it, type it, use it, break it, fix it

The pieces on AWS

Piece	Job
ECS Fargate	Runs the container; no servers to manage
Task definition	The recipe: image, CPU/memory, env vars, secrets, logs
Service	Keeps N copies running, registers them with the load balancer
Application Load Balancer	Public entry point; routes to healthy containers only
RDS	Managed PostgreSQL

The **network** (VPC, subnets) already exists. We only read it.

🧪 Lab — Start here

Goal: starter copied, credentials working, `terraform.tfvars` created with a handle nobody else picked.

⏱ 7 minutes

The password nobody types

```
resource "aws_db_instance" "orders" {
  username                    = "app"
  manage_master_user_password = true
}
```

- RDS **generates** the password and stores it in **Secrets Manager**
- Not in the code, not in `.tfvars`, not in the plan, **not in the state**
- Consumers reference the secret's ARN; ECS fetches the value when the container starts

🧪 Lab — Step 1

Goal: start creating your database. It takes 5–10 minutes, so leave that terminal running and open your editor.

⏱ 5 minutes

Part 1

Designing the module

A module is a promise

```
inputs  ──►  [ web-service ]  ──►  outputs
name, image, port,                   url,
tier, env, secrets,                  security group,
network                              service name, logs
```

- The **interface** is what every caller lives with. Write it **first**
- Inside can change freely; the interface can't, without breaking callers
- Good inputs are **decisions a caller should make** — nothing more

Opinionated inputs beat raw knobs

```
variable "size" {
  type = string
  validation {
    condition     = contains(["small", "medium", "large"], var.size)
    error_message = "size must be small, medium or large."
  }
}
```

- Callers pick a **size**; the module maps it to numbers it knows are valid
- Fewer ways to get it wrong, one place to change what "medium" means
- `validation` turns a bad value into a clear error at `plan`, not a failure at `apply`

Maps or lists of objects?

```
env = [                                          env = {
  { name = "LOG_LEVEL", value = "info" },          LOG_LEVEL = "info"
  { name = "REGION", value = "eu" },               REGION    = "eu"
]                                                }
```

- Both work. The map **can't** have duplicate keys, and it's half the typing
- Maps **merge**: defaults + overrides with `merge(a, b)`
- The module converts to whatever shape the API wants — once, in `locals`

🧪 Lab — Step 2 (type along)

Goal: the module's `versions.tf` and `variables.tf`, then `terraform validate` says it's valid.

⏱ 10 minutes

Part 2

Locals do the translating

Expressions you'll use today

```
[for k, v in var.labels : { key = k, value = v }]   # map → list of objects
var.kms_key == null ? "AES256" : "aws:kms"          # conditional
distinct([for q in values(var.queues) : q.region])  # unique values
jsonencode({ retries = 3 })                          # HCL → JSON string
merge(var.tags, { Name = "orders-db" })              # later wins
```

- A `for` over a **map** runs in **sorted key order** → stable plans
- Keep translation in `locals`; keep resources short and readable

`dynamic` blocks: zero or more of a nested block

```
dynamic "ingress" {
  for_each = var.allow_ssh ? [22] : []
  content {
    from_port = ingress.value
    to_port   = ingress.value
  }
}
```

- `for_each` over an empty list → the block disappears
- Use it when a block really must be optional or repeated; otherwise write it out

🧪 Lab — Step 3 (type along)

Goal: the task definition, the service and the execution role.
`validate` should complain **only** about things Step 4 creates.

⏱ 13 minutes

The security-group chain

```
internet —80—▶ [ ALB SG ] —container port—▶ [ service SG ] —443—▶ internet
```

- Each hop needs a rule on **both** sides: out of one group, into the next
- The service accepts traffic **only from its own load balancer**
- Rules reference **security groups**, not IP ranges — they survive IP changes

🧪 Lab — Step 4 (type along)

Goal: load balancer, target group, listener, both security groups, and the outputs. `validate` says valid.

⏱ 10 minutes

Calling your module

```
module "orders" {
  source = "./modules/web-service"

  name  = "orders-api"
  image = "example/orders:1.4.2"
  tier  = "small"
  # plus container_port, environment, secrets,
  # and where it runs: cluster_arn, vpc_id, subnet_ids
}
```

- Everything app-specific is an **input**; the module stays generic
- **Not** everything is a module: the cluster (shared) and the database (app-specific) stay outside
- A new module → `terraform init` before `plan`

🧪 Lab — Step 5

Goal: deploy Wiki.js with your module, wired to your database, and open it.

⏱ 10 minutes

🧪 Lab — Step 6

Goal: get a `200` from your wiki URL.

⏱ 15 minutes

A 502/503 is a symptom. Logs are the evidence.

- **502/503** from the load balancer = "no working target" — *where*, not *why*
- The container log said it **timed out** reaching its database; `DB_HOST` / `DB_PORT` said where
- Security groups: a new connection needs **egress** where it starts and **ingress** where it lands
- The module allowed only 443 out on purpose — and handed you the SG to extend

🧪 Lab — Step 7

Goal: set up your wiki and write a page. Then change the module's `tier` and read the plan — don't apply.

⏱ 8 minutes

Update in place, or replace?

- The plan says which: `~` update in place, `-/+` replace
- Task definitions are **immutable**: every change → new revision (replace)
- The service is **updated** to point at the new revision; ECS rolls the containers
- Your wiki page survived: it lives in the database, not the container

What production would add

Here	Production
Local state	S3 backend with `use_lockfile`, shared by the team and CI
HTTP on port 80	HTTPS with an ACM certificate, 80 → 443 redirect
Tasks in public subnets	Private subnets + NAT gateway or VPC endpoints
`PGSSLMODE=no-verify`	RDS CA certificate in `DB_SSL_CA` (verified TLS)
Module in a folder	Module in its own repo, tagged versions, `?ref=v1.2.0`
No backups, no deletion protection	Both on, plus monitoring and alarms

Key takeaways

- Design the **interface first**: what should a caller decide?
- **Opinionated inputs** (sizes, maps) prevent mistakes that raw knobs allow
- **Locals translate**; resources stay short
- **Secure by default, extensible on purpose** (443-only egress + the SG output)
- **Managed secrets**: the best secret in state is the one that was never there
- Debug from **evidence**: logs, then the path the packet takes

🧪 Teardown

`terraform destroy` — the database takes a few minutes to delete.

Own-account people: destroy `network/` afterwards.

⏱ 10 minutes (questions while it runs)